République Tunisienne
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Université de Tunis El Manar

Faculté des Sciences de Tunis

Cycle ingénieur en génie logiciel

Département des Sciences de l'informatique

Projet NoSQL

Migration Base Relationnelle

MongoDB | Apache Cassandra | Neo4j

Base de Données : Gestion de Bibliothèque

Réalisé par : Belhareh Ines, Asma Raies

Encadré par : Houda Alaya

Année Universitaire : 2025-2026

Table des matières

1 Introduction **3**
1.1 Objectifs du projet . . . 3
1.2 Récapitulatif des 20 requêtes avancées . . . 3

2 Base de Données Relationnelle (SQLite) **5**
2.1 Schéma Entité-Relation . . . 5
2.1.1 Diagramme ER (schéma simplifié) . . . 5
2.2 Relations entre les tables . . . 6
2.3 Extrait du script SQL . . . 6
2.4 Données de la base (extrait) . . . 7

3 Migration vers MongoDB **8**
3.1 Stratégie de migration . . . 8
3.2 Structure d'un document — Collection livres . . . 8
3.3 Requêtes MongoDB . . . 9
3.4 Résultats des requêtes . . . 9

4 Migration vers Apache Cassandra **10**
4.1 Stratégie Query-Driven Design . . . 10
4.2 Création du Keyspace . . . 10
4.3 Schéma des tables principales . . . 10
4.4 Requêtes SELECT en CQL . . . 11

5 Requêtes Avancées MongoDB **13**
5.1 M1 — Statistiques par Catégorie . . . 13
5.2 M2 — $lookup : Jointure Livres et Emprunts . . . 13
5.3 M3 — $facet : Analyse Multi-Dimensionnelle . . . 14
5.4 M4 — Score d'Activité des Membres . . . 14
5.5 M5 — Valeur du Stock par Auteur . . . 15
5.6 M6 — Activité Mensuelle . . . 15
5.7 M7 — Recommandation Collaborative . . . 16
5.8 M8 — Rang et Percentile par Catégorie . . . 16
5.9 M9 — Détection d'Anomalies . . . 16
5.10 M10 — KPI Dashboard . . . 17

6 Requêtes Avancées Apache Cassandra **18**
6.1 C1 — COUNTER TABLE . . . 18
6.2 C2 — TTL (Time To Live) . . . 18
6.3 C3 — LWT (Lightweight Transactions) . . . 19
6.4 C4 — MATERIALIZED VIEW . . . 19
6.5 C5 — SASI Index . . . 20
6.6 C6 — BATCH . . . 20
6.7 C7 — Time-Series avec Bucketing . . . 20
6.8 C8 — UDA (User Defined Aggregates) . . . 21
6.9 C9 — Bucketing Anti Hot-Partition . . . 21
6.10 C10 — Consistency Levels (CAP) . . . 22

7 Migration vers Neo4j **23**
7.1 Pourquoi un graphe pour une bibliothèque ? . . . 23
7.2 Modèle du graphe . . . 23
7.2.1 Nœuds et propriétés . . . 23
7.2.2 Relations et cardinalités . . . 24
7.2.3 Schéma du graphe (diagramme) . . . 24
7.3 Contraintes et index . . . 24
7.4 Création des nœuds et relations . . . 25
7.5 Requêtes Cypher (G1–G10) . . . 26
7.6 Avantages du modèle graphe . . . 29

8 Comparaison des Quatre Approches **30**
8.1 Guide de choix technologique . . . 30
8.2 Synthèse des concepts avancés . . . 30

9 Guide d'Installation et d'Exécution **32**
9.1 Prérequisites . . . 32
9.2 Étapes d'exécution . . . 32

10 Conclusion **34**
10.1 Bilan par approche . . . 34
10.2 Règle de décision . . . 34

1 Introduction

Ce projet illustre la migration d'une base de données relationnelle vers trois systèmes **NoSQL** de nature différente : **MongoDB** (base orientée documents), **Cassandra** (base orientée colonnes *wide-column*) et **Neo4j** (base orientée graphe). La base choisie modélise un **système de gestion de bibliothèque** comprenant des livres, des auteurs, des membres et des emprunts.

L'objectif est d'analyser les différences structurelles entre le modèle relationnel et les modèles NoSQL, et de comprendre les stratégies de migration adaptées à chaque paradigme. Le document présente **28 requêtes avancées** illustrant les fonctionnalités expertes de chaque système.

1.1 Objectifs du projet

Étapes du projet

Étape	Description
Modèle relationnel	Concevoir et peupler une base SQLite normalisée (3NF)
Migration MongoDB	Transformer en documents JSON dénormalisés et imbriqués
Migration Cassandra	Modéliser selon les requêtes cibles (Query-Driven Design)
Migration Neo4j	Modéliser en nœuds et relations (Graph-Driven Design)
Requêtes avancées	Illustrer les fonctionnalités expertes de chaque système
Comparaison	Analyser les avantages et inconvénients de chaque approche

1.2 Récapitulatif des 20 requêtes avancées

ID	Requête	Concept NoSQL	Niveau
M1	Stats par catégorie	Aggregation + $group + $avg	
M2	Livres empruntés actifs	$lookup (jointure)	
M3	Analyse multi-dimensionnelle	$facet + $bucket	
M4	Score d'activité membre	$addFields + formule	
M5	Valeur stock par auteur	$group + $multiply	
M6	Activité mensuelle	Pipeline temporel	

ID	Requête	Concept NoSQL	Niveau
M7	Recommandation collaborative	Intersection d'ensembles	
M8	Rang et percentile	$setWindowFields + $rank	
M9	Détection d'anomalies	$match + taux de retard	
M10	KPI Dashboard	Pipeline multi-étapes	
C1	Compteurs atomiques	COUNTER TABLE	
C2	Sessions avec expiration	TTL (Time To Live)	
C3	Réservations conditionnelles	LWT — protocole Paxos	
C4	Vue synchronisée	MATERIALIZED VIEW	
C5	Recherche textuelle	SASI Index (LIKE)	
C6	Écriture multi-tables	BATCH (logged)	
C7	Historique temporel	Time-Series + bucketing	
C8	Agrégat personnalisé	UDA (User Defined Aggregate)	
C9	Équilibrage des partitions	Bucketing anti hot-partition	
C10	Choix de cohérence	Consistency Levels (CAP)	
G5	Détection de retards	Filtre sur relation	
G6	Recommandations collaboratives	Intersection de chemins	
G8	Centralité des livres	Degré de connexion	
G9	Réseau de lecture	Membres connectés	
G10	Stats par catégorie	Agrégation graphe	

Table 1.1 – Récapitulatif des 28 requêtes avancées

2 Base de Données Relationnelle (SQLite)

La base relationnelle est conçue selon les règles de normalisation (3NF). Elle comporte **6 tables** interconnectées par des clés étrangères.

2.1 Schéma Entité-Relation

Table	Attributs principaux	Nb
`categories`	id, nom, description	5
`auteurs`	id, nom, prenom, nationalite, date_naissance	8
`livres`	id, titre, isbn, annee_pub, editeur, prix, auteur_id, categorie_id	10
`membres`	id, nom, prenom, email, telephone, ville, date_inscription	8
`exemplaires`	id, livre_id, etat, disponible	20
`emprunts`	id, membre_id, exemplaire_id, date_emprunt, date_retour_prevue, statut	8

Table 2.1 – Résumé des tables de la base relationnelle

2.1.1 Diagramme ER (schéma simplifié)

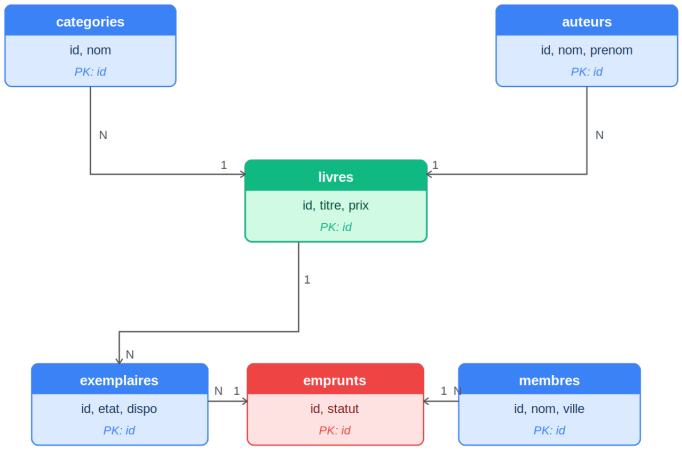


Figure 2.1 – Diagramme Entité-Relation de la base Bibliothèque

2.2 Relations entre les tables

Table source	Table cible	Clé étrangère	Cardinalité
livres	auteurs	auteur_id → auteurs.id	N :1
livres	categories	categorie_id → categories.id	N :1
exemplaires	livres	livre_id → livres.id	N :1
emprunts	membres	membre_id → membres.id	N :1
emprunts	exemplaires	exemplaire_id → exemplaires.id	N :1

TABLE 2.2 – Relations (clés étrangères) entre les tables

2.3 Extrait du script SQL

Listing 2.1 – DDL SQL des tables principales

```sql
CREATE TABLE livres (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    titre         TEXT NOT NULL,
    isbn          TEXT UNIQUE,
    annee_pub     INTEGER,
    editeur       TEXT,
    prix          REAL,
    auteur_id     INTEGER REFERENCES auteurs(id),
    categorie_id INTEGER REFERENCES categories(id)
);

CREATE TABLE emprunts (
    id                  INTEGER PRIMARY KEY AUTOINCREMENT,
    membre_id           INTEGER REFERENCES membres(id),
    exemplaire_id       INTEGER REFERENCES exemplaires(id),
    date_emprunt        TEXT NOT NULL,
    date_retour_prevue TEXT NOT NULL,
    date_retour_reelle TEXT,
    statut TEXT CHECK(statut IN ('en_cours','rendu','retard'))
);
```

2.4 Données de la base (extrait)

ID	Titre	Auteur	Catégorie	Prix	Éditeur
1	The Art of Computer Programming	Knuth	Informatique	89.99 TND	Addison-Wesley
2	Clean Code	Martin	Informatique	45.00 TND	Prentice Hall
3	The Relational Model for DB	Codd	Informatique	55.00 TND	Addison-Wesley
4	L'Étranger	Camus	Littérature	12.00 TND	Gallimard
5	Modern Operating Systems	Tanenbaum	Informatique	70.00 TND	Pearson
6	Refactoring	Fowler	Informatique	50.00 TND	Addison-Wesley
7	Cent ans de solitude	García M.	Littérature	14.00 TND	Gallimard
8	Ainsi parlait Zarathoustra	Nietzsche	Philosophie	11.00 TND	Gallimard

Table 2.3 – Table livres (8 premières lignes sur 10)

3 Migration vers MongoDB

MongoDB — Base orientée documents (JSON/BSON)

MongoDB stocke les données sous forme de documents BSON. La stratégie adoptée est la **dénormalisation** : les données liées sont regroupées dans un seul document, évitant les jointures coûteuses.

3.1 Stratégie de migration

Principe	Détail
Dénormalisation	Auteur et catégorie imbriqués dans le document livre
Sous-documents	Les exemplaires deviennent un tableau de sous-documents
Historique imbriqué	Les emprunts sont stockés dans l'exemplaire concerné
Champs calculés	`nb_exemplaires` et `nb_disponibles` précalculés
2 collections	`livres` (dénormalisé) + `membres` (enrichi)

Table 3.1 – Principes de la migration vers MongoDB

3.2 Structure d'un document — Collection livres

Listing 3.1 – Document MongoDB du livre « Clean Code »

```
{
  "_id": 2,
  "titre": "Clean Code",
  "annee_pub": 2008,
  "editeur": "Prentice Hall",
  "prix": 45.0,
  "auteur": { "nom": "Martin", "prenom": "Robert C." },
  "categorie": { "nom": "Informatique" },
  "nb_exemplaires": 2,
  "nb_disponibles": 1,
  "exemplaires": [
    { "exemplaire_id": 3, "etat": "bon", "disponible": true },
    {
      "exemplaire_id": 4, "etat": "neuf", "disponible": false,
      "historique_emprunts": [
        { "emprunt_id": 3, "membre_id": 3,
          "date_emprunt": "2024-01-20", "statut": "en_cours" }
      ]
```

```
    }
  ]
}
```

3.3 Requêtes MongoDB

3.3.0.1 Livres d'informatique

```
db.livres.find({"categorie.nom": "Informatique"},
               {"titre": 1, "auteur.nom": 1})
```

3.3.0.2 Membres en retard

```
db.membres.find({"emprunts_retard": {"$gt": 0}},
                {"nom": 1, "prenom": 1, "emprunts_retard": 1})
```

3.3.0.3 Livres par catégorie

```
db.livres.aggregate([
  {"$group": {"_id": "$categorie.nom", "count": {"$sum": 1}}},
  {"$sort":  {"count": -1}}
])
```

3.3.0.4 Mettre à jour un exemplaire

```
db.livres.updateOne(
  {"exemplaires.exemplaire_id": 3},
  {"$set": {"exemplaires.$.disponible": true}}
)
```

3.4 Résultats des requêtes

Requête	Résultats	Nb
Livres d'informatique	Clean Code, Modern OS, Refactoring, Art of CP...	6
Livres disponibles	Tous les 10 livres (au moins 1 ex. dispo)	10
Membres en retard	Trabelsi Youssef (1), Sassi Amira (1)	2
Livres/catégorie	Informatique :6, Littérature :3, Philosophie :1	3

Table 3.2 – Résultats des principales requêtes MongoDB

4 Migration vers Apache Cassandra

Cassandra — Base wide-column distribuée (CQL)

Cassandra n'offre **pas de jointures ni de requêtes ad-hoc**. La modélisation doit être guidée par les **requêtes cibles** (Query-Driven Design).

4.1 Stratégie Query-Driven Design

ID	Requête cible	Table	Partition Key	Clustering Key
Q1	Livres par catégorie	`livres_par_categorie`	categorie	titre, livre_id
Q2	Livres d'un auteur	`livres_par_auteur`	auteur_nom, prenom	titre
Q3	Emprunts d'un membre	`emprunts_par_membre`	membre_id	date_emprunt DESC
Q4	Disponibilité	`disponibilite_livres`	livre_id	exemplaire_id
Q5	Emprunts en retard	`emprunts_en_retard`	statut	date_retour_prevue
Q6	Membres par ville	`membres_par_ville`	ville	nom, prenom

Table 4.1 – Correspondance requêtes → tables Cassandra

4.2 Création du Keyspace

Listing 4.1 – Keyspace avec réplication simple (facteur 3)

```
CREATE KEYSPACE IF NOT EXISTS bibliotheque
WITH REPLICATION = {
    'class': 'SimpleStrategy',
    'replication_factor': 3
}
AND DURABLE_WRITES = true;

USE bibliotheque;
```

4.3 Schéma des tables principales

Q1 — livres_par_categorie

```
CREATE TABLE livres_par_categorie (
```

```
    categorie       TEXT,
    titre           TEXT,
    livre_id        INT,
    prix            DECIMAL,
    auteur_nom      TEXT,
    auteur_prenom   TEXT,
    nb_disponibles  INT,
    PRIMARY KEY ((categorie), titre, livre_id)
) WITH CLUSTERING ORDER BY (titre ASC);
```

Q3 — emprunts_par_membre

```
CREATE TABLE emprunts_par_membre (
    membre_id           INT,
    date_emprunt        TEXT,
    emprunt_id          INT,
    livre_titre         TEXT,
    date_retour_prevue  TEXT,
    statut              TEXT,
    PRIMARY KEY ((membre_id), date_emprunt, emprunt_id)
) WITH CLUSTERING ORDER BY (date_emprunt DESC);
```

Q5 — emprunts_en_retard

```
CREATE TABLE emprunts_en_retard (
    statut              TEXT,
    date_retour_prevue  TEXT,
    emprunt_id          INT,
    membre_nom          TEXT,
    livre_titre         TEXT,
    PRIMARY KEY ((statut), date_retour_prevue, emprunt_id)
) WITH CLUSTERING ORDER BY (date_retour_prevue ASC);
```

4.4 Requêtes SELECT en CQL

4.4.0.1 🔍 Livres d'informatique

```
SELECT titre, auteur_nom, auteur_prenom, nb_disponibles
FROM livres_par_categorie
WHERE categorie = 'Informatique';
```

4.4.0.2 🔍 Emprunts en retard

```
SELECT membre_nom, livre_titre, date_retour_prevue
FROM emprunts_en_retard
```

```
WHERE statut = 'retard';
```

4.4.0.3 🔍 Membres à Tunis

```
SELECT nom, prenom, email
FROM membres_par_ville
WHERE ville = 'Tunis';
```

5 Requêtes Avancées MongoDB

Fonctionnalités Avancées MongoDB (M1–M10)

Aggregation Pipeline, $lookup, $facet, $setWindowFields, détection d'anomalies et KPI Dashboard.

5.1 M1 — Statistiques par Catégorie

Listing 5.1 – M1 — Agrégation par catégorie

```
db.livres.aggregate([
  { $group: {
      _id: "$categorie.nom",
      nb_livres:         { $sum: 1 },
      prix_moyen:        { $avg: "$prix" },
      total_exemplaires: { $sum: "$nb_exemplaires" },
      total_disponibles: { $sum: "$nb_disponibles" }
  }},
  { $addFields: {
      taux_dispo: {
        $divide: ["$total_disponibles", "$total_exemplaires"]
      }
  }},
  { $sort: { nb_livres: -1 }}
])
```

Résultat : Prix moyen Informatique = 61.66 TND, taux disponibilité = 66.7%.

5.2 M2 — $lookup : Jointure Livres et Emprunts

Listing 5.2 – M2 — $lookup inter-collections

```
db.livres.aggregate([
  { $unwind: "$exemplaires" },
  { $lookup: {
      from: "membres",
```

```
      let: { m_id:
         "$exemplaires.historique_emprunts.membre_id" },
      pipeline: [
        { $match: { $expr: { $in: ["$_id", "$$m_id"] }}}
      ],
      as: "membre_info"
  }},
  { $match: { "exemplaires.disponible": false }},
  { $project: { titre: 1, "membre_info.nom": 1 }}
])
```

5.3 M3 — $facet : Analyse Multi-Dimensionnelle

Listing 5.3 – M3 — 4 dimensions en une seule requête

```
db.livres.aggregate([{ $facet: {
  "par_categorie": [
    { $group: { _id: "$categorie.nom", count: { $sum: 1 }}}
  ],
  "par_decade": [{ $bucket: {
      groupBy: "$annee_pub",
      boundaries: [1880, 1950, 1970, 1990, 2010, 2030],
      default: "Autre"
  }}],
  "prix_distribution": [{ $bucket: {
      groupBy: "$prix",
      boundaries: [0, 15, 30, 50, 75, 100]
  }}]
}}])
```

5.4 M4 — Score d'Activité des Membres

$$\text{score} = (\text{emprunts} \times 10) + (\text{rendus} \times 5) - (\text{retards} \times 20)$$

Listing 5.4 – M4 — Score et profil membre

```
db.membres.aggregate([
  { $addFields: {
      score_activite: {
        $subtract: [
          { $add: [
              { $multiply: ["$nb_emprunts_total", 10] },
              { $multiply: ["$emprunts_rendus",    5] }
          ]},
```

```
          { $multiply: ["$emprunts_retard", 20] }
        ]},
      profil: { $switch: { branches: [
        { case: { $gte: ["$score_activite", 30] },
          then: "Excellent" },
        { case: { $gte: ["$score_activite", 10] },
          then: "Bon" }
      ], default: "Mauvais" }}
  }},
  { $sort: { score_activite: -1 }}
])
```

5.5 M5 — Valeur du Stock par Auteur

Listing 5.5 – M5 — Valeur financière du stock

```
db.livres.aggregate([
  { $group: {
      _id: "$auteur.nom",
      valeur_stock: {
        $sum: { $multiply: ["$prix", "$nb_exemplaires"] }
      },
      prix_moyen: { $avg: "$prix" }
  }},
  { $sort: { valeur_stock: -1 }}
])
```

5.6 M6 — Activité Mensuelle

Listing 5.6 – M6 — Pipeline temporel

```
db.membres.aggregate([
  { $unwind: "$emprunts" },
  { $addFields: {
      mois: { $dateToString: {
        format: "%Y-%m",
        date: { $dateFromString: {
          dateString: "$emprunts.date_emprunt"
        }}
      }}
  }},
  { $group: {
      _id: "$mois",
      nb_emprunts: { $sum: 1 }
```

```
  }},
  { $sort: { _id: 1 }}
])
```

5.7 M7 — Recommandation Collaborative

Listing 5.7 – M7 — Co-occurrences de lectures

```
db.membres.aggregate([
  { $unwind: "$emprunts" },
  { $group: {
      _id: "$emprunts.livre_titre",
      lecteurs: { $addToSet: "$_id" }
  }},
  { $addFields: {
      nb_communs: { $size: {
        $setIntersection: ["$lecteurs", "$$autres.lecteurs"]
      }}
  }},
  { $match: { nb_communs: { $gt: 0 }}},
  { $sort: { nb_communs: -1 }}, { $limit: 3 }
])
```

5.8 M8 — Rang et Percentile par Catégorie

Listing 5.8 – M8 — $setWindowFields (MongoDB 5.0+)

```
db.livres.aggregate([
  { $setWindowFields: {
      partitionBy: "$categorie.nom",
      sortBy: { prix: -1 },
      output: {
        rang_prix:  { $rank: {} },
        percentile: { $percentRank: {} }
      }
  }},
  { $project: {
      titre: 1, "categorie.nom": 1,
      prix: 1, rang_prix: 1
  }}
])
```

5.9 M9 — Détection d'Anomalies

Listing 5.9 – M9 — Membres à risque

```
db.membres.aggregate([
  { $addFields: {
      taux_retard: {
        $cond: [
          { $gt: ["$nb_emprunts_total", 0] },
          { $divide: ["$emprunts_retard",
                      "$nb_emprunts_total"] },
          0
        ]}
  }},
  { $match: { taux_retard: { $gte: 0.5 }}},
  { $project: { nom:1, prenom:1, taux_retard:1 }}
])
```

Alerte Anomalies

Résultat : 2 membres avec 100% de taux de retard détectés.

5.10 M10 — KPI Dashboard

Listing 5.10 – M10 — Indicateurs clés

```
db.livres.aggregate([
  { $group: {
      _id: null,
      total_livres:      { $sum: 1 },
      valeur_stock:      { $sum: { $multiply:
                           ["$prix","$nb_exemplaires"]}},
      total_disponibles: { $sum: "$nb_disponibles"},
      livres_non_empruntes: {
        $sum: { $cond:
          [{ $eq: ["$nb_emprunts_total",0]}, 1, 0]}
      }
  }}
])
```

KPI	Valeur	KPI	Valeur
Total livres	10	**Total exemplaires**	20
Valeur stock	833.98 TND	**Disponibles**	10/20 (50%)
Non empruntés	5	**Taux retard**	25%

TABLE 5.1 – Résultat M10 — KPI Dashboard

6 Requêtes Avancées Apache Cassandra

Fonctionnalités Avancées Cassandra (C1–C10)

COUNTER, TTL, LWT, Materialized Views, SASI, BATCH, Time-Series, UDA, Bucketing et Consistency Levels.

6.1 C1 — COUNTER TABLE

Listing 6.1 – C1 — COUNTER atomique

```
CREATE TABLE compteurs_livres (
    livre_id           INT PRIMARY KEY,
    nb_emprunts_total  COUNTER,
    nb_consultations   COUNTER
);

-- Incrementer sans lire la valeur avant :
UPDATE compteurs_livres
SET nb_emprunts_total = nb_emprunts_total + 1
WHERE livre_id = 2;

SELECT livre_id, nb_emprunts_total
FROM compteurs_livres WHERE livre_id = 2;
```

6.2 C2 — TTL (Time To Live)

Listing 6.2 – C2 — Expiration automatique

```
CREATE TABLE sessions_membres (
    session_id TEXT PRIMARY KEY,
    membre_id  INT,
    ip_adresse TEXT,
    created_at TIMESTAMP
);

```

```
-- Session expire apres 24h :
INSERT INTO sessions_membres
    (session_id, membre_id, ip_adresse, created_at)
VALUES ('sess_abc123', 3, '192.168.1.1', toTimestamp(now()))
USING TTL 86400;

SELECT session_id, TTL(ip_adresse) AS ttl_restant
FROM sessions_membres
WHERE session_id = 'sess_abc123';
```

6.3 C3 — LWT (Lightweight Transactions)

Listing 6.3 – C3 — Réservation sans double-booking

```
-- Reserver SEULEMENT si disponible :
INSERT INTO reservations
    (exemplaire_id, membre_id, statut)
VALUES (5, 3, 'en_attente')
IF NOT EXISTS;
-- [applied: true]  si succes
-- [applied: false] si deja reserve

UPDATE reservations SET statut = 'confirmee'
WHERE exemplaire_id = 5
IF statut = 'en_attente';
```

Coût des LWT

Latence 4× supérieure (4 round-trips Paxos). À utiliser uniquement si la cohérence forte est indispensable.

6.4 C4 — MATERIALIZED VIEW

Listing 6.4 – C4 — Vue auto-synchronisée

```
CREATE MATERIALIZED VIEW livres_par_categorie_prix AS
    SELECT * FROM livres_base
    WHERE categorie IS NOT NULL
      AND prix IS NOT NULL
      AND livre_id IS NOT NULL
PRIMARY KEY ((categorie), prix, livre_id)
WITH CLUSTERING ORDER BY (prix ASC);

SELECT titre, prix FROM livres_par_categorie_prix
WHERE categorie = 'Informatique' AND prix <= 50;
```

6.5 C5 — SASI Index

Listing 6.5 – C5 — Recherche LIKE avec SASI

```
CREATE CUSTOM INDEX idx_titre_sasi
ON livres_recherche (titre)
USING 'org.apache.cassandra.index.sasi.SASIIndex'
WITH OPTIONS = { 'mode': 'CONTAINS',
                 'case_sensitive': 'false' };

SELECT titre, prix FROM livres_recherche
WHERE titre LIKE '%Code%';
```

6.6 C6 — BATCH

Listing 6.6 – C6 — Écriture atomique multi-tables

```
BEGIN BATCH
    INSERT INTO emprunts_par_membre
        (membre_id, date_emprunt, emprunt_id,
         livre_titre, statut)
    VALUES (3, '2024-04-01', 9, 'Clean␣Code', 'en_cours');

    UPDATE disponibilite_livres
    SET disponible = false
    WHERE livre_id = 2 AND exemplaire_id = 4;

    UPDATE compteurs_livres
    SET nb_emprunts_total = nb_emprunts_total + 1
    WHERE livre_id = 2;
APPLY BATCH;
```

6.7 C7 — Time-Series avec Bucketing

Listing 6.7 – C7 — Bucketing mensuel

```
CREATE TABLE historique_activite (
    annee_mois TEXT,     -- '2024-03'
    livre_id   INT,
    event_time TIMESTAMP,
    event_type TEXT,     -- emprunt / retour
    membre_id  INT,
```

```
    PRIMARY KEY ((annee_mois, livre_id), event_time)
) WITH CLUSTERING ORDER BY (event_time DESC)
  AND default_time_to_live = 31536000;

SELECT event_time, event_type, membre_id
FROM historique_activite
WHERE annee_mois = '2024-03' AND livre_id = 2 LIMIT 1;
```

6.8 C8 — UDA (User Defined Aggregates)

Listing 6.8 – C8 — Agrégat personnalisé en Java

```
CREATE OR REPLACE FUNCTION state_count(
    state MAP<TEXT, INT>, val TEXT
) CALLED ON NULL INPUT
RETURNS MAP<TEXT, INT>
LANGUAGE java AS '
    if (val != null) {
        Integer n = state.get(val);
        state.put(val, (n == null ? 0 : n) + 1);
    }
    return state;
';

CREATE AGGREGATE count_by_value(TEXT)
SFUNC state_count
STYPE MAP<TEXT, INT>
INITCOND {};

SELECT count_by_value(statut) FROM emprunts_par_membre
WHERE membre_id = 1;
-- {'en_cours': 2, 'rendu': 1, 'retard': 1}
```

6.9 C9 — Bucketing Anti Hot-Partition

Listing 6.9 – C9 — Distribution uniforme

```
-- MAUVAIS : toute la ville sur 1 noeud
CREATE TABLE membres_mauvais (
    ville TEXT, membre_id INT, nom TEXT,
    PRIMARY KEY (ville, membre_id)
);

-- BON : bucket = membre_id % 10
```

```sql
CREATE TABLE membres_par_ville_bucket (
    ville TEXT, bucket INT,
    membre_id INT, nom TEXT, prenom TEXT,
    PRIMARY KEY ((ville, bucket), membre_id)
);

SELECT * FROM membres_par_ville_bucket
WHERE ville = 'Tunis'
  AND bucket IN (0,1,2,3,4,5,6,7,8,9);
```

6.10 C10 — Consistency Levels (CAP)

Listing 6.10 – C10 — Niveaux de cohérence (RF=3)

```sql
-- ONE : 1 replique, disponibilite max
CONSISTENCY ONE;
SELECT titre FROM livres_par_categorie
WHERE categorie = 'Informatique';

-- QUORUM : 2 repliques, coherence forte
CONSISTENCY QUORUM;
INSERT INTO emprunts_par_membre (...) VALUES (...);

-- ALL : 3 repliques, coherence absolue
CONSISTENCY ALL;
UPDATE disponibilite_livres SET disponible = false
WHERE livre_id = 1 AND exemplaire_id = 1;
```

Opération	Level	Justification
Lecture catalogue	ONE	Données peu critiques
Enregistrer emprunt	QUORUM	Cohérence forte requise
Marquer indisponible	ALL	Opération critique
Multi-datacenter	LOCAL_QUORUM	Minimise la latence

Table 6.1 – Guide du niveau de cohérence par opération

7 Migration vers Neo4j

Neo4j — Base de Données Orientée Graphe (Cypher)

Neo4j est la base de données graphe la plus répandue. Elle stocke les données sous forme de **nœuds** et de **relations**, ce qui la rend naturellement adaptée aux structures hiérarchiques, aux réseaux sociaux et aux systèmes de recommandation.

7.1 Pourquoi un graphe pour une bibliothèque ?

Une bibliothèque est un **réseau de relations** : un auteur *écrit* un livre, un livre *appartient* à une catégorie, un membre *emprunte* un exemplaire. Dans le modèle relationnel ces liens sont matérialisés par des clés étrangères ; dans Neo4j ils deviennent des **citoyens de première classe**, traversables en une seule requête Cypher.

Cas d'usage typiques de Neo4j

Recommandations	« Les membres qui ont lu X ont aussi lu Y »
Détection de fraude	Chemin suspect entre entités
Réseau social	Degré de connexion, clustering
Analyse de chemins	Plus court chemin entre deux nœuds
Knowledge Graph	Ontologies et graphes de connaissances

7.2 Modèle du graphe

7.2.1 Nœuds et propriétés

Label	Propriétés principales	Nb	Contrainte
`:Livre`	livre_id, titre, isbn, prix, annee_pub	10	UNIQUE(livre_id)
`:Auteur`	auteur_id, nom, prenom, nationalite	8	UNIQUE(auteur_id)
`:Membre`	membre_id, nom, prenom, email, ville	8	UNIQUE(membre_id)
`:Categorie`	nom, description	5	UNIQUE(nom)
`:Exemplaire`	exemplaire_id, etat, disponible	20	UNIQUE(exemplaire_id)
TOTAL nœuds		**51**	

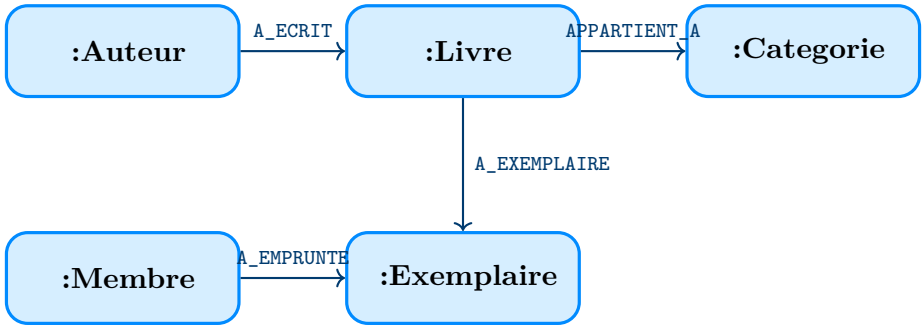
Table 7.1 – Nœuds du graphe Bibliothèque

7.2.2 Relations et cardinalités

De	Type	Vers	Propriétés	Nb
:Auteur	-[:A_ECRIT]->	:Livre	—	10
:Livre	-[:APPARTIENT_A]->	:Categorie	—	10
:Livre	-[:A_EXEMPLAIRE]->	:Exemplaire	—	20
:Membre	-[:A_EMPRUNTE]->	:Exemplaire	date_emprunt, statut	8
TOTAL relations				**48**

TABLE 7.2 – Relations du graphe avec leurs propriétés

7.2.3 Schéma du graphe (diagramme)



A_EMPRUNTE : {emprunt_id, date_emprunt, date_retour_prevue, statut}

FIGURE 7.1 – Modèle du graphe Neo4j — Bibliothèque (51 nœuds, 48 relations)

7.3 Contraintes et index

Listing 7.1 – Contraintes d'unicité et index de recherche

```
// Contraintes d'unicite (cree aussi un index B-tree)
CREATE CONSTRAINT IF NOT EXISTS FOR (l:Livre)
  REQUIRE l.livre_id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (a:Auteur)
  REQUIRE a.auteur_id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (m:Membre)
  REQUIRE m.membre_id IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (c:Categorie)
  REQUIRE c.nom IS UNIQUE;
CREATE CONSTRAINT IF NOT EXISTS FOR (e:Exemplaire)
  REQUIRE e.exemplaire_id IS UNIQUE;

// Index supplementaires pour la recherche
CREATE INDEX IF NOT EXISTS FOR (l:Livre)  ON (l.titre);
```

```
CREATE INDEX IF NOT EXISTS FOR (m:Membre) ON (m.ville);
CREATE INDEX IF NOT EXISTS FOR (m:Membre) ON (m.email);
```

7.4 Création des nœuds et relations

Listing 7.2 – Création des nœuds principaux

```
// Noeud Categorie
MERGE (:Categorie {nom: 'Informatique',
  description: 'Livres␣sur␣la␣programmation␣et␣les␣systemes'});

// Noeud Auteur
MERGE (:Auteur {auteur_id: 2, nom: 'Martin', prenom: 'Robert',
  nationalite: 'Americain', date_naissance: '1952-12-05'});

// Noeud Livre
MERGE (:Livre {livre_id: 2, titre: 'Clean␣Code',
  isbn: '978-0-13-235088-4', annee_pub: 2008,
  editeur: 'Prentice␣Hall', prix: 45.0});

// Noeud Membre
MERGE (:Membre {membre_id: 2, nom: 'Chaabane', prenom:
   'Fatima',
  email: 'f.chaabane@email.com', ville: 'Sfax'});

// Noeud Exemplaire
MERGE (:Exemplaire {exemplaire_id: 3, etat: 'use',
  disponible: false});
```

Listing 7.3 – Création des relations

```
// (Auteur)-[:A_ECRIT]->(Livre)
MATCH (a:Auteur {auteur_id: 2}), (l:Livre {livre_id: 2})
MERGE (a)-[:A_ECRIT]->(l);

// (Livre)-[:APPARTIENT_A]->(Categorie)
MATCH (l:Livre {livre_id: 2}), (c:Categorie {nom:
   'Informatique'})
MERGE (l)-[:APPARTIENT_A]->(c);

// (Livre)-[:A_EXEMPLAIRE]->(Exemplaire)
MATCH (l:Livre {livre_id: 2}), (ex:Exemplaire {exemplaire_id:
   3})
MERGE (l)-[:A_EXEMPLAIRE]->(ex);

```

```
// (Membre)-[:A_EMPRUNTE {props}]->(Exemplaire)
MATCH (m:Membre {membre_id: 2}),
      (ex:Exemplaire {exemplaire_id: 3})
MERGE (m)-[:A_EMPRUNTE {
  emprunt_id: 2,
  date_emprunt: '2024-03-05',
  date_retour_prevue: '2024-03-19',
  date_retour_reelle: null,
  statut: 'en_cours'
}]->(ex);
```

7.5 Requêtes Cypher (G1–G10)

G1 — Livres d'une catégorie

```
MATCH (l:Livre)-[:APPARTIENT_A]->(c:Categorie {nom:
   'Informatique'})
MATCH (a:Auteur)-[:A_ECRIT]->(l)
RETURN l.titre AS titre,
       a.nom + '␣' + a.prenom AS auteur,
       l.prix AS prix
ORDER BY l.titre;
```

Résultat G1 — 6 livres d'informatique

```
Clean Code / Martin Robert / 45.0 TND
Modern Operating Systems / Tanenbaum Andrew / 70.0 TND
Patterns of Enterprise App Arch / Fowler Martin / 60.0 TND
Refactoring / Fowler Martin / 50.0 TND
The Art of Computer Programming / Knuth Donald / 89.99 TND
The Relational Model for DB / Codd Edgar / 55.0 TND
```

G2 — Livres d'un auteur avec sa catégorie

```
MATCH (a:Auteur {nom: 'Fowler'})-[:A_ECRIT]->(l:Livre)
      -[:APPARTIENT_A]->(c:Categorie)
RETURN l.titre AS titre,
       l.annee_pub AS annee,
       c.nom AS categorie
ORDER BY l.annee_pub DESC;
```

G3 — Emprunts actifs (chemin complet)

```
MATCH (m:Membre)-[e:A_EMPRUNTE]->(ex:Exemplaire)
      <-[:A_EXEMPLAIRE]-(l:Livre)
```

```
WHERE e.statut IN ['en_cours', 'retard']
RETURN m.prenom + '␣' + m.nom AS membre,
       l.titre AS livre,
       e.date_emprunt AS depuis,
       e.statut AS statut
ORDER BY e.date_emprunt;
```

Membre	Livre	Depuis	Statut
Youssef Trabelsi	The Relational Model for DB	2024-02-20	retard
Amira Sassi	Clean Code	2024-02-28	retard
Fatima Chaabane	Clean Code	2024-03-05	en_cours
Leila Mansouri	Modern Operating Systems	2024-03-10	en_cours
Sofien Mejri	The Relational Model for DB	2024-03-15	en_cours
Nadia Khelifi	L'Étranger	2024-03-18	en_cours

TABLE 7.3 – Résultat G3 — Emprunts actifs

G5 — Détection des retards

```
MATCH (m:Membre)-[e:A_EMPRUNTE]->(ex:Exemplaire)
      <-[:A_EXEMPLAIRE]-(l:Livre)
WHERE e.statut = 'retard'
RETURN m.prenom + '␣' + m.nom AS membre,
       l.titre AS livre,
       e.date_retour_prevue AS prevue,
       m.email AS contact
ORDER BY e.date_retour_prevue;
```

⚠ Résultat G5 — 2 emprunts en retard

Youssef Trabelsi — The Relational Model for DB — prévu le 2024-03-05
Amira Sassi — Clean Code — prévu le 2024-03-13

G6 — Recommandations collaboratives

```
// "Les membres qui ont lu X ont aussi lu Y"
MATCH (m:Membre)-[:A_EMPRUNTE]->(:Exemplaire)
      <-[:A_EXEMPLAIRE]-(l1:Livre),
      (m)-[:A_EMPRUNTE]->(:Exemplaire)
      <-[:A_EXEMPLAIRE]-(l2:Livre)
WHERE l1 <> l2
RETURN DISTINCT l1.titre AS livre_source,
                l2.titre AS recommandation,
                count(DISTINCT m) AS lecteurs_communs
```

```
ORDER BY lecteurs_communs DESC
LIMIT 10;
```

G8 — Centralité et popularité des livres

```
MATCH (l:Livre)<-[:A_EXEMPLAIRE]-(ex:Exemplaire)
OPTIONAL MATCH (ex)<-[e:A_EMPRUNTE]-(:Membre)
WITH l,
     count(DISTINCT ex) AS nb_exemplaires,
     count(DISTINCT e)  AS nb_emprunts
RETURN l.titre AS titre,
       nb_exemplaires,
       nb_emprunts,
       round(nb_emprunts * 1.0 / nb_exemplaires, 2) AS taux
ORDER BY nb_emprunts DESC;
```

Titre	Exemplaires	Emprunts	Taux
The Art of Computer Programming	2	2	1.00
Clean Code	2	2	1.00
The Relational Model for DB	2	2	1.00
L'Étranger	2	1	0.50
Modern Operating Systems	2	1	0.50
Refactoring, Cent ans de solitude, ...	2	0	0.00

TABLE 7.4 – Résultat G8 — Centralité des livres (taux d'utilisation)

G9 — Réseau de lecture entre membres

```
// Membres partageant des lectures communes
MATCH (m1:Membre)-[:A_EMPRUNTE]->(:Exemplaire)
      <-[:A_EXEMPLAIRE]-(l:Livre)
      <-[:A_EXEMPLAIRE]-(:Exemplaire)<-[:A_EMPRUNTE]-(m2:Membre)
WHERE m1 <> m2 AND id(m1) < id(m2)
RETURN m1.prenom + ' ' + m1.nom AS membre1,
       m2.prenom + ' ' + m2.nom AS membre2,
       collect(DISTINCT l.titre) AS livres_communs,
       count(DISTINCT l) AS nb_livres_communs
ORDER BY nb_livres_communs DESC;
```

Résultat G9 — 3 paires de membres connectées

Membre 1	Membre 2	Livre commun
Mohamed Ben Ali	Karim Brahmi	The Art of Computer Programming
Fatima Chaabane	Amira Sassi	Clean Code
Youssef Trabelsi	Sofien Mejri	The Relational Model for DB

G10 — Statistiques par catégorie (graphe)

```
MATCH (c:Categorie)<-[:APPARTIENT_A]-(l:Livre)
      <-[:A_EXEMPLAIRE]-(ex:Exemplaire)
OPTIONAL MATCH (ex)<-[e:A_EMPRUNTE]-(:Membre)
WITH c.nom AS categorie,
     count(DISTINCT l)  AS nb_livres,
     count(DISTINCT ex) AS nb_exemplaires,
     count(DISTINCT e)  AS nb_emprunts,
     avg(l.prix)        AS prix_moyen,
     sum(l.prix * 2)    AS valeur_stock
RETURN categorie, nb_livres, nb_exemplaires,
       nb_emprunts, round(prix_moyen, 2) AS prix_moyen,
       valeur_stock
ORDER BY nb_livres DESC;
```

Catégorie	Livres	Emprunts	Prix moyen	Valeur stock
Informatique	6	7	61.66 TND	739.98 TND
Littérature	3	1	12.00 TND	72.00 TND
Philosophie	1	0	11.00 TND	22.00 TND

TABLE 7.5 – Résultat G10 — Agrégation par catégorie (graphe)

7.6 Avantages du modèle graphe

Pourquoi Neo4j excelle ici

Relations natives	Un JOIN SQL devient une simple traversée de flèche
Requêtes intuitives	Cypher lit comme une phrase : `(m)-[:A_EMPRUNTE]->(l)`
Recommandations	L'intersection de chemins est naturelle en graphe
Performance	Traversée en O(1) par relation, indépendant du volume
Évolutivité	Ajouter un type de relation sans modifier le schéma

8 Comparaison des Quatre Approches

Critère	SQLite	MongoDB	Cassandra	Neo4j
Modèle	Tables (2D)	Documents JSON	Wide-column	Nœuds + Relations
Schéma	Fixe (DDL)	Flexible	Fixe par table	Flexible (labels)
Langage	SQL	MQL / Aggregation	CQL	Cypher
Jointures	Oui (JOIN)	$lookup	Non	Traversée native
Scalabilité	Verticale	Horizontale	Horizontale (massive)	Verticale/Cluster
Cohérence	ACID	Éventuelle	Éventuelle (tunable)	ACID
Avantage	ACID, simplicité	Flexibilité	Écriture scalable	Relations complexes
Inconvénient	Scale limité	Pas de TX multi-doc	Pas de jointures	Scale horiz. limité
Cas d'usage	Petites apps	Web, IoT, mobile	Big Data, haute dispo	Réseau, reco, fraude
Collections	6 tables	2 collections	6 tables	51 nœuds / 48 rels

Table 8.1 – Comparaison SQLite vs MongoDB vs Apache Cassandra vs Neo4j

8.1 Guide de choix technologique

Si votre besoin est...	Choisissez
Intégrité stricte, app modeste	SQLite / PostgreSQL
Données hiérarchiques, lectures fréquentes	MongoDB
Volume massif, haute disponibilité	Cassandra
Réseaux, recommandations, chemins	Neo4j

Table 8.2 – Guide de choix selon le cas d'usage

8.2 Synthèse des concepts avancés

Concepts MongoDB

Aggregation Pipeline	`$group`, `$match`, `$sort`, `$project`, `$unwind`
$lookup	Jointure inter-collections
$facet	Analyse multi-dimensionnelle
$setWindowFields	Rang et percentile (v5.0+)
Anomaly Detection	Comportements anormaux
KPI Dashboard	Pipeline complet

Concepts Cassandra

COUNTER	Compteurs atomiques distribués
TTL	Expiration automatique des données
LWT	Transactions conditionnelles (Paxos)
Mat. View	Vue secondaire auto-synchronisée
SASI	Recherche textuelle `LIKE`
BATCH	Écriture atomique multi-tables
Time-Series	Bucketing temporel
UDA	Agrégats personnalisés (Java)
Anti Hot-Partition	Bucketing modulo
Consistency Levels	ONE, QUORUM, ALL

Concepts Neo4j

Traversée native	Chemin `(m)-[:REL]->(n)` sans jointure
Pattern matching	Reconnaissance de sous-graphes
Shortest Path	`shortestPath()` entre deux nœuds
Centralité	Degré de connexion des nœuds
Reco. collaborative	Intersection de chemins de lecture
Réseau de lecture	Membres connectés par livres communs

9 Guide d'Installation et d'Exécution

9.1 Prérequisites

Outil	Installation / URL
Python 3.8+	`pip install pymongo cassandra-driver neo4j`
MongoDB	https://www.mongodb.com/try/download/community
Cassandra	https://cassandra.apache.org/download/
Neo4j	https://neo4j.com/download/ (ports 7687 / 7474)
SQLite	Inclus dans Python (`import sqlite3`)

Table 9.1 – Prérequisites et installation

9.2 Étapes d'exécution

Étape 1. Créer la base SQLite

```
python 01_base_relationnelle.py
```

Étape 2. Générer les documents MongoDB

```
python 02_migration_mongodb.py
```

Étape 3. Générer le schéma Cassandra

```
python 03_migration_cassandra.py
```

Étape 4. Générer le schéma Neo4j

```
python 04_migration_neo4j.py
```

Étape 5. Insérer dans MongoDB

```
mongoimport --db bibliotheque --collection livres \
  --file mongo_livres.json --jsonArray
```

Étape 6. Exécuter le CQL (Cassandra)

```
cqlsh -f schema_cassandra.cql
```

Étape 7. Charger dans Neo4j

```
# Via cypher-shell :
cypher-shell -u neo4j -p motdepasse -f schema_neo4j.
   cypher

# Via Neo4j Browser (http://localhost:7474) :
# Copier-coller le contenu de schema_neo4j.cypher
```

Étape 8. Explorer le graphe (Neo4j Browser)

```
// Visualiser tout le graphe
MATCH (n)-[r]->(m) RETURN n, r, m LIMIT 50;

// Sous-graphe d'un livre
MATCH path = (a:Auteur)-[:A_ECRIT]->
  (l:Livre {titre: 'Clean Code'})-[:A_EXEMPLAIRE]->(ex)
RETURN path;
```

10 Conclusion

10.1 Bilan par approche

SQLite

Idéal pour les petites applications nécessitant une intégrité stricte. La conformité **ACID** reste son atout majeur.

MongoDB

Excelle pour les données hiérarchiques et les lectures fréquentes. L'**Aggregation Pipeline** offre des capacités analytiques avancées.

Apache Cassandra

Conçu pour la scalabilité massive et la haute disponibilité. Le **Query-Driven Design** garantit des performances constantes à grande échelle.

Neo4j

Idéal pour les données fortement connectées : recommandations, réseaux sociaux et détection de fraude. Le langage **Cypher** rend les requêtes traversant plusieurs relations aussi lisibles qu'une phrase en langage naturel.

10.2 Règle de décision

Le choix d'un système **NoSQL** doit être guidé par les **besoins fonctionnels**, les **patterns d'accès**, les contraintes de **scalabilité** et la **complexité** des requêtes. La maîtrise de plusieurs systèmes (**polyglot persistence**) est la compétence clé de l'ingénieur data moderne.

Références

— MongoDB Documentation — https://docs.mongodb.com

— Apache Cassandra Documentation — https://cassandra.apache.org/doc/
— Neo4j Documentation — https://neo4j.com/docs/
— Fowler & Sadalage — *NoSQL Distilled* (2012), Addison-Wesley
— Bradshaw et al. — *MongoDB : The Definitive Guide* (2019), O'Reilly
— Carpenter & Hewitt — *Cassandra : The Definitive Guide* (2022), O'Reilly
— Robinson et al. — *Graph Databases* (2015), O'Reilly
— SQLite Documentation — https://www.sqlite.org/docs.html